

Agentic Natural Language Querying Across Structured Database

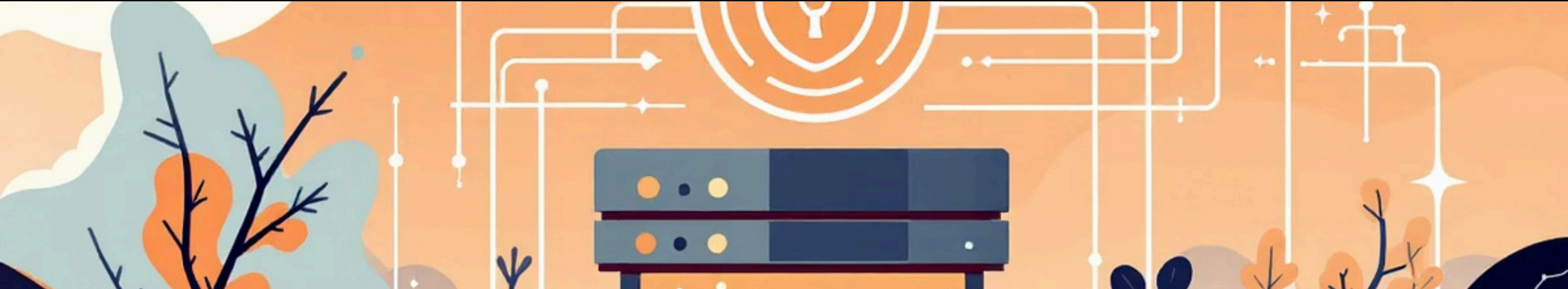
Members :

Shreyas Mohite

Shubham Naik

Rutuja Kadam





Problem Definition

Traditional single-agent Text-to-SQL systems often fall short in complex, real-world scenarios.

Lack of Robustness

Vulnerable to errors, struggling with complex query planning and semantic understanding.

Inadequate Error Recovery

Limited ability to self-correct or provide meaningful feedback on failed queries.

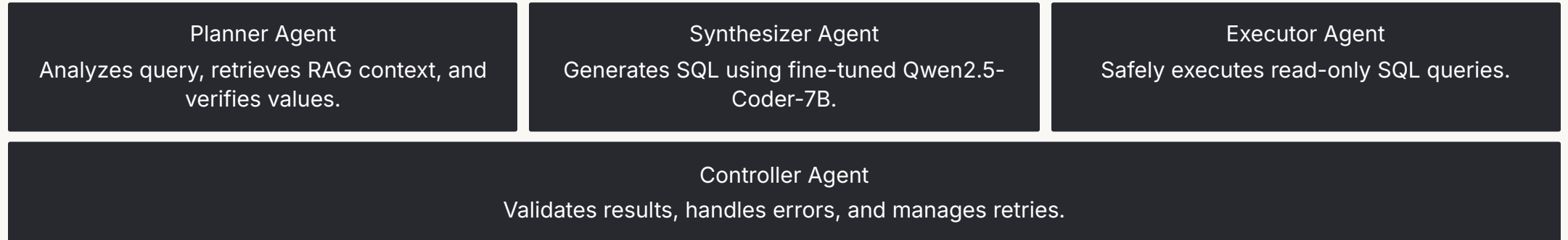
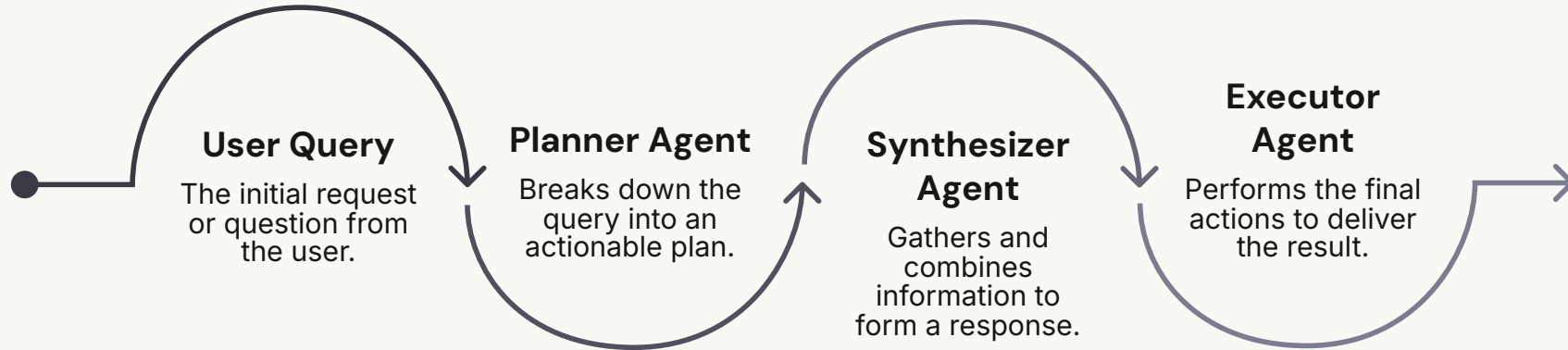
Critical Safety Concerns

Absence of mechanisms to prevent destructive operations like DROP, DELETE or TRUNCATE

Our Goal: Building a Secure, Accurate, and Robust Text-to-SQL System

Methodology : Multi-Agent Architecture for Enhanced Reliability

Leveraging the LangGraph framework, we developed a system with specialized agents for precision and safety.



RAG-enhanced context with ChromaDB provides crucial information from schema cards, glossary, and query cookbooks.

Dataset Description

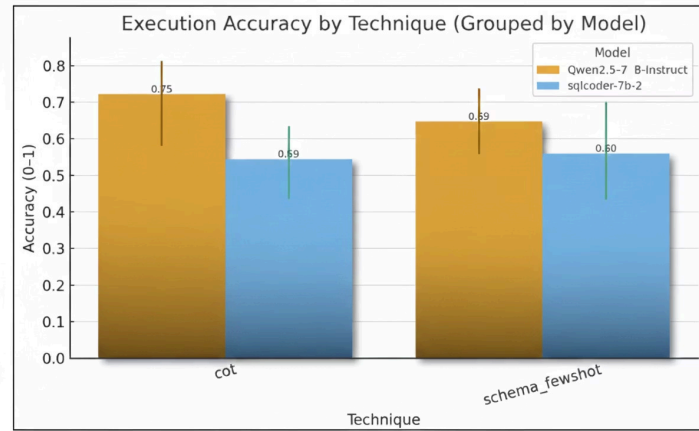
Spider Dataset

- Used for LoRA Fine tuning Qwen2.5-Coder-7B-Instruct model.
- 9000 questions, where 5,693 complex SQL queries across 200+ databases.
- Used as benchmark for Text-to-SQL tasks
- We tested on 2000 examples.

Chinook Database

- Used for multi-agent demonstration and pipeline testing
- The **Chinook dataset** is a sample music store database with about **7,700 total records** across **11 tables**.
- Generated schema cards for semantic understanding. Created query cookbook with verified examples, glossary
- tested on 150 user queries.

Preprocessing Steps and Experimental Design:



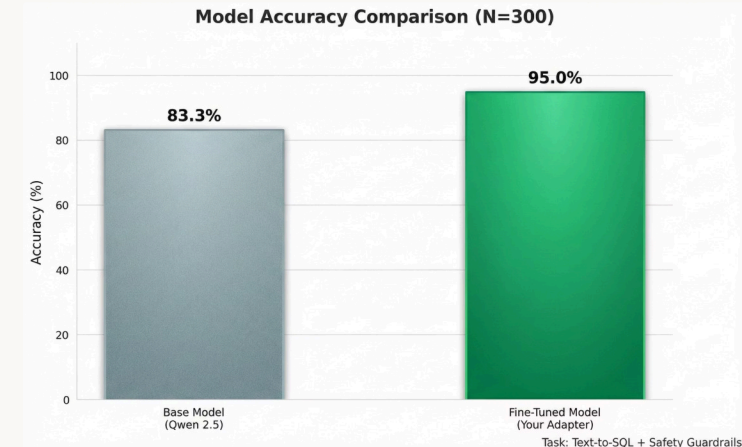
Prompt Engineering:

Structured planning prompts with dynamic schema context.

Incorporation of SQLite-specific rules (e.g., `||` for concatenation, `strftime` for dates).

Integration of error feedback for iterative retry attempts.

Enforcement of JSON schema for reliable QueryPlan outputs.



Model Fine-tuning:

Base Model: Qwen2.5-Coder-7B-Instruct, chosen after comparative analysis of multiple models and prompting techniques.

Method: LoRA (Low-Rank Adaptation) PEFT adapter for efficient SQL synthesis.

Dataset: Spider dataset, renowned for its complexity and diversity in Text-to-SQL tasks.

Experimental Design:

RAG Context: Enabling Semantic Understanding

Retrieval-Augmented Generation (RAG) is crucial for grounding our model in specific database schemas and business logic.



Dynamic Schema Extraction

Utilizing SQLite PRAGMA commands for accurate and up-to-date schema information.



RAG Document Sources

Schema cards with foreign key relationships, a comprehensive business glossary, and a query cookbook.



ChromaDB Vector Indexing

Facilitating semantic retrieval of relevant context for query generation.



Pydantic Models

Ensuring structured outputs from LLMs for robust integration.

notebook

Results:

```
=====
Question: How many tracks are in the database?
=====
```

```
The following generation flags are not valid and may be ignored: ['temperature', 'top_p', 'top_k']. Set `TRANSFORMERS_VERBOSITY=info` for more details.
```

```
[DEBUG] Event: {'retrieve_context': {'schema_context': 'Table: Track\nColumn: Composer\nData type: NVARCHAR(220)\nIs PK: False\nFK ref: nan\nSample: Angus Young, Malcolm Y
```

```
🔍 [Planner] Retrieved context from RAG
```

```
[DEBUG] Plan created successfully: Track
```

```
[DEBUG] Event: {'analyze_and_plan': {'query_plan': {'primary_entity': 'Track', 'tables_involved': ['Track'], 'join_path': 'None', 'filters': [], 'aggregations': 'COUNT', 'ordering': None, 'limit': 50, 'error_message': None, 'error_feedback': None}}
```

```
📋 [Planner] Created plan: Track from ['Track']
```

```
[DEBUG] Event: {'synthesizer': {'generated_sql': 'SELECT COUNT(*) FROM TRACK LIMIT 50', 'status': 'executing'}}
```

```
📝 [Synthesizer] Generated SQL:
```

```
SELECT COUNT(*) FROM TRACK LIMIT 50
```

```
[DEBUG] Event: {'executor': {'execution_result': {'success': True, 'columns': ['COUNT(*)'], 'data': [(3503,)], 'row_count': 1, 'total_row_count': 1, 'truncated': False, 'error_message': None, 'error_feedback': None}}
```

```
✂ [Executor] Query returned 1 rows
```

```
[DEBUG] Event: {'controller': {'final_answer': 'There are 3503 tracks in the database.', 'status': 'complete'}}
```

```
💬 [Controller] Generated final answer
```

```
[DEBUG] Final state keys: ['user_question', 'schema_context', 'glossary_context', 'cookbook_context', 'query_plan', 'needs_clarification', 'clarification_request', 'resolved']
```

```
[DEBUG] Status: complete
```

```
[DEBUG] Query plan: {'primary_entity': 'Track', 'tables_involved': ['Track'], 'join_path': 'None', 'filters': [], 'aggregations': 'COUNT', 'ordering': None, 'limit': 50, 'error_message': None, 'error_feedback': None}
```

```
[DEBUG] Generated SQL: SELECT COUNT(*) FROM TRACK LIMIT 50
```

```
[DEBUG] Execution result: {'success': True, 'columns': ['COUNT(*)'], 'data': [(3503,)], 'row_count': 1, 'total_row_count': 1, 'truncated': False, 'error_message': None, 'error_feedback': None}
```

```
[DEBUG] Error feedback: None
```

```
=====
ANSWER:
=====
```

```
There are 3503 tracks in the database.
```

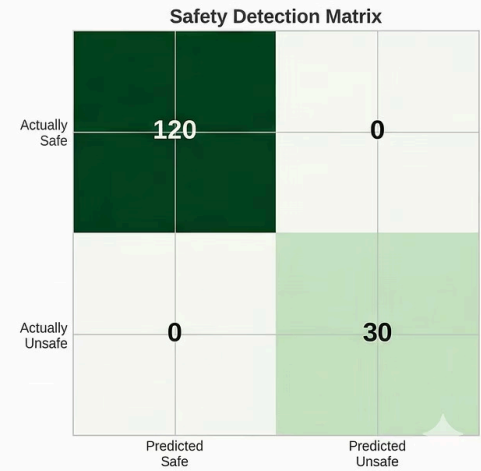
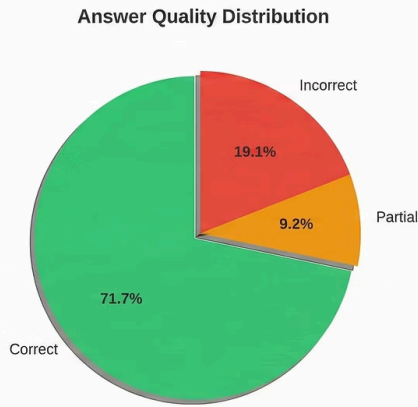
```
=====
Question: delete the data with the country has the highest total sales?
=====

[Planner] Retrieved context from RAG
[Planner] analyze_and_plan output: <class 'dict'>

=====
ANSWER:
=====
I'm sorry, but I can only help with SELECT queries to retrieve data from the database. I cannot perform destructive operations like DELETE.
```

Match (Partial):

Question: "List 5 rock tracks" Gold: SELECT Name FROM Track WHERE GenrelId=1 LIMIT 5 → Returns 5 rows
Generated: SELECT Name FROM Track WHERE GenrelId=1 LIMIT 10 → Returns 10 rows (contains the 5)



Results: Performance and Safety Validation

98%

Safety Detection

Blocked all destructive queries with zero false positives.

71.7%

Accuracy

Correctly generated SQL aligned with user intent.

80.9%

Overall Success

Combined rate of accurate and executable queries.



Conclusion & Future Work

We have built a robust and secure Text-to-SQL system, with clear paths for future enhancements.



Achievements

- 100% safety detection against destructive queries.
- 80.9% of the time, the user receives a useful, executable result from the database.
- Effective separation of concerns through multi-agent design.



Future Work

- Support for more complex nested queries and Common Table Expressions (CTEs).
- Implementation of interactive clarification for ambiguous user queries.
- Human in the loop for Planner and executor agent.



Potential Applications

- Transforming enterprise database interfaces for broader accessibility.
- Enhancing Business Intelligence (BI) tools with natural language capabilities.